

Class 6: R functions

Tonhi Dinh (PID: 18508020)

Table of contents

Background:	1
Our first function	1
A second function	3
A protein generating function	5

Background:

All functions in R have at least 3 things:

- A **name** that we use to call the function.
- One or more input **arguments**
- The **body** the lines of R code tat do the work

Our first function

Let's write a silly wee function call `add()` to add some numbers (the input arguments)

```
add <- function(x, y){  
  x + y  
}
```

Now we can use this function

```
add(100,1)
```

```
[1] 101
```

```
add(x=10, y=10)
```

```
[1] 20
```

```
add(x=c(100, 1, 100), y=1)
```

```
[1] 101    2 101
```

Q. What if I give a multiple element vector to x and y?

```
add(x=c(100,1), y=c(100,1))
```

```
[1] 200    2
```

Q. What if I give three inputs to the function?

```
#add(x=c(100,1), y=1, z=1)
```

Q. What if I give only one input to the add function?

```
addnew <- function(x, y=1){  
  x + y  
}
```

```
addnew(x=100)
```

```
[1] 101
```

```
addnew(c(100,1), 100)
```

```
[1] 200 101
```

If we write our function with input arguments having no default value then the user will be required to set them when they use the function. We can give our argument default value by setting them equal to some sensible value - e.g. y=1 in the `addnew()` function

A second function

Let's try something more interesting: Make a sequence generating tool..

The `sample()` function can be a useful starting point

```
sample(1:10, size=4)
```

```
[1] 10  3  9  6
```

Q. Generate 9 random numbers taken from the input vector `x=1:10`?

```
sample(1:10, size = 9)
```

```
[1]  8  9  7  5 10  3  2  4  6
```

Q. Generate 12 random numbers taken from the input vector `x=1:10`?

```
sample(1:10, size = 12, replace = TRUE)
```

```
[1]  1  6  8  1  7  8  6  9  8 10  4  9
```

Q. Write code for the `sample()` function that generates nucleotide sequences of length 6

```
sample(x=c("A", "T", "C", "G"), size=6, replace=TRUE)
```

```
[1] "C" "A" "G" "T" "T" "G"
```

Q. Write a first function `generate_dna()` that return a user specified length DNA sequence:

```
generate_dna <- function(len){  
  sample(c("A", "T", "C", "G"), replace = TRUE, size = len)  
}
```

```
generate_dna(len=100)
```

```
[1] "C" "G" "A" "T" "G" "A" "T" "A" "C" "C" "T" "A" "C" "T" "C" "C" "A" "A"
[19] "A" "T" "G" "C" "A" "T" "A" "G" "T" "C" "G" "T" "G" "A" "G" "G" "C" "G"
[37] "C" "G" "A" "T" "A" "C" "T" "C" "A" "C" "A" "T" "C" "A" "T" "G" "C" "C"
[55] "A" "A" "G" "T" "A" "T" "C" "A" "G" "C" "A" "G" "T" "T" "G" "A" "C" "A"
[73] "A" "A" "G" "T" "C" "G" "A" "G" "G" "G" "G" "T" "T" "G" "T" "A" "T" "G"
[91] "C" "T" "C" "G" "G" "T" "G" "G" "A" "C"
```

Key-Points Every function in R looks fundamentally the same in terms of its structure. Basically 3 things: name, input, and body

```
name <- function(input){ body }
```

Functions can have multiple inputs. These can be **required** arguments or **optional** arguments. With optional arguments having a set default value

Modify and improve our `generate_dna()` function to return its generated sequence in a more standard format like “AGTAGC” rather than the vector “A”, “C”, “G”, “A”

```
generate_dna <- function(len=6, fasta=TRUE){
  ans <- sample(c("A", "T", "C", "G"),
               replace = TRUE, size = len)
  if(fasta){
    cat("Single-element vector output")
    ans <- paste(ans, collapse = "")
  } else{
    cat("Multi-element vector output")
  }
  return(ans)
}
generate_dna(fasta=FALSE)
```

Multi-element vector output

```
[1] "T" "G" "T" "T" "C" "G"
```

```
generate_dna(fasta=TRUE)
```

Single-element vector output

```
[1] "GCGTAG"
```

The `paste()` function - it's job is to join up or stick together (a.k.a. paste) input strings together

```
paste(c("alice", "barry"), "loves R", sep=" ")
```

```
[1] "alice loves R" "barry loves R"
```

Flow control means where the R brain goes in your code

```
good_mood <- FALSE

if(good_mood){
  cat("Great!")
} else{
  cat("Bummer!")
}
```

Bummer!

A protein generating function

Q. Write a function that generates a user specified length protein sequence.

```
generate_protein <- function(len){
  # The amino acids to sample from
  aa <- c("A", "C", "D", "E", "F", "G", "H",
          "I", "K", "L", "M", "N", "P", "Q",
          "R", "S", "T", "V", "W", "Y")

  # Draw n=len amino acids to make our sequence
  ans <- sample(aa, replace = TRUE, size = len)
  ans <- paste(ans, collapse = "")
  return(ans)
}
my_seq <- generate_protein(42)
```

Q. Use that function to generate random protein sequences between length 6 and 12

```
generate_protein(len=6)
```

```
[1] "FWWVKC"
```

```
generate_protein(len=7)
```

```
[1] "NCDLMWE"
```

```
generate_protein(len=8)
```

```
[1] "GIDGFKHH"
```

```
generate_protein(len=9)
```

```
[1] "HHMWWHYVV"
```

```
generate_protein(len=10)
```

```
[1] "HHDSNMEIHV"
```

```
generate_protein(len=11)
```

```
[1] "PWNVNPSFFMF"
```

```
generate_protein(len=12)
```

```
[1] "DNLAPMSWTPRD"
```

```
for (i in 6:12) {  
  # FASTA ID line ">id"  
  cat(">", i, sep="", "\n")  
  # Our random protein sequence line  
  cat( generate_protein(i), "\n" )  
}
```

>6
EEPKWQ
>7
CINQMSE
>8
TFDRNNGS
>9
WQHGYACKC
>10
HTSYWFMIEQ
>11
YIKRMMACCQD
>12
CTNGHRSHYTPF

Q. Are any of your sequences unique i.e. not found anywhere in nature?

All of the sequences shown below are unique >8 LHMSWVHS >9 TQQHYGMAI >10 TVHI-
WEFGFR >11 YMCLLMEVHNL >12 NRMFFAQVDSNK